

Project Report: Distributed Object Store with Integrity Verification

1. Introduction

This project is a distributed storage system that keeps files safe even when storage nodes crash or silently corrupt data. Instead of storing three full copies of every file (the usual approach), we use erasure coding to split each file into 6 fragments and spread them across 6 separate storage nodes. Any 4 of those 6 fragments are enough to get the original file back, so the system can tolerate 2 nodes going down at the same time — at only 1.5x the storage cost instead of 3x.

But tolerating missing fragments is only half the problem. We also need to know when a fragment has been tampered with or corrupted. For that, we implemented the Fingerprinted Cross-Checksum (FPCC) from Hendricks, Ganger, and Reiter’s 2007 PODC paper. The FPCC is a small proof that lets us verify each fragment independently — without needing to download all the other fragments to compare. If a fragment fails verification, we just treat it like a missing one and reconstruct from the rest.

On top of that, we implemented the paper’s AVID-FP protocol, which adds a server-to-server consensus layer so that even a malicious node can’t trick the system into accepting bad data.

Everything is written in Java 17 with no external libraries — the erasure coding, the finite field math, the fingerprinting, the networking — all hand-coded using just the standard library.

2. Why This Approach

The standard way to make data fault-tolerant is replication: store 3 copies. It works, but it’s expensive. For a 1 TB dataset, you need 3 TB of storage.

Erasure coding does better. Our (6, 4) code gives the same level of fault tolerance (survive any 2 failures) but only uses 1.5 TB for that same 1 TB dataset. The tradeoff is that encoding and decoding require some computation, but the storage savings are worth it.

Now, simply hashing each fragment can tell you if something changed, but it can’t tell you whether the fragments are *consistent with each other* — i.e., whether they all came from encoding the same original file. An attacker could swap in a fragment from a different file that passes its own hash check but produces garbage when decoded alongside the others. The FPCC solves this with a fingerprinting function that respects the linear algebra of erasure coding: you can check any single fragment against a compact proof without touching the others.

3. How It Works

3.1 GF(2⁸) Arithmetic

Everything — the erasure codec and the fingerprinting — runs over GF(2⁸), the finite field with 256 elements. Each element fits in a single byte. Addition is XOR. Multiplication uses the irreducible polynomial $x^8 + x^4 + x^3 + x^2 + 1$ (0x11D) and a shift-and-reduce loop. Inverses come from Fermat’s Little Theorem: $a^{-1} = a^{254}$. We also implemented full Gauss-Jordan matrix inversion over this field, which the decoder needs to recover data from arbitrary subsets of fragments.

All of this lives in `GaloisField256.java` (96 lines).

3.2 Erasure Coding

The codec is a (6, 4) systematic Vandermonde code. The coding matrix looks like:

```
[ 1 0 0 0 ] ← identity (data rows)
[ 0 1 0 0 ]
[ 0 0 1 0 ]
[ 0 0 0 1 ]
[ 1 1 1 1 ] ← parity row (x=1)
[ 1 2 4 8 ] ← parity row (x=2)
```

The top 4 rows are the identity, so the first 4 fragments are just the original data chunks — no transformation needed. The bottom 2 rows generate parity fragments by multiplying the data through Vandermonde-style coefficients in $GF(2^8)$.

To decode, we pick any 4 surviving fragments, pull out the matching 4 rows of the coding matrix, invert that 4x4 sub-matrix over $GF(2^8)$, and multiply. The Vandermonde structure guarantees this inversion always works for any 4-subset.

Implemented in `LinearErasureCodec.java` (166 lines).

3.3 Fingerprinted Cross-Checksum (FPCC)

The FPCC has three parts: - **cc[0..5]**: SHA-256 hash of each encoded fragment. - **fp[0..3]**: Homomorphic fingerprint of each source (data) fragment. - **seed**: Derived as SHA-256 of all the hashes concatenated together (acts as a random oracle).

The fingerprint treats each fragment's bytes as coefficients of a polynomial over $GF(2^8)$ and evaluates it at random points derived from the seed. The key property: if a parity fragment is a linear combination of data fragments (which it is, by construction), then its fingerprint is the same linear combination of the data fingerprints. So we can compute what the fingerprint *should* be for any fragment — data or parity — using only the data fingerprints and the coding matrix.

To verify fragment i , we check two things: 1. Does SHA-256 of the fragment match `cc[i]`? 2. Does the actual fingerprint match the expected one computed from `fp[]` and the coding row?

If either check fails, we treat the fragment as an erasure. This is the same code path as a crashed node — simple and uniform.

Implemented across `FingerprintedCrossChecksum.java` (98 lines) and `HomomorphicFingerprint.java` (111 lines).

3.4 AVID-FP Protocol

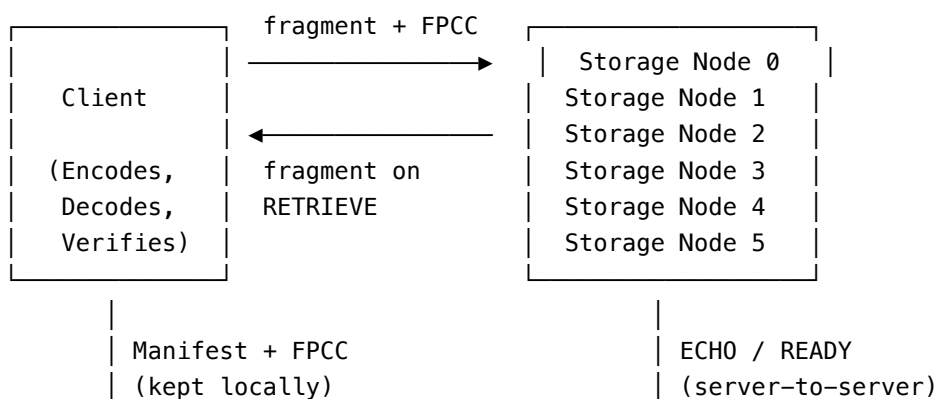
This is the consensus protocol from Section 4 of the paper. It uses echo/ready quorums to make sure enough servers agree before considering data “delivered.” The parameters are $n=6$ servers, $m=4$ source fragments, $f=1$ Byzantine fault tolerance.

The flow: 1. Client sends each server its fragment + the full FPCC. 2. Each server verifies its fragment's hash, stores it, and broadcasts ECHO to all peers. 3. When a server collects $m+f=5$ echoes, it broadcasts READY. 4. When a server gets $f+1=2$ READY messages, it also sends READY (amplification). 5. When a server has $2f+1=3$ READY messages, it marks the data as delivered and tells the client.

For retrieval, the client fetches from all servers, groups responses by FPCC, finds the FPCC that at least $f+1=2$ servers agree on, verifies fragments against it, and decodes.

Implemented in `AvidFpClient.java` (243 lines) on the client side and the AVID-FP handlers in `StorageNode.java`.

4. Architecture



Storage nodes are simple: they store byte arrays, retrieve them, and optionally verify incoming data against attached FPCC data. They communicate over plain TCP sockets, one request per connection. In AVID-FP mode, they also talk to each other for echo/ready consensus.

The client does all the heavy lifting: encoding, FPCC computation, parallel upload/download, verification, and decoding. It saves a manifest file locally that records which node holds which fragment, the erasure coding parameters, and the full FPCC. This manifest never leaves the client machine — that’s the trust model. Nodes can verify what they’re given, but they can’t forge a valid FPCC.

Two modes of operation: - upload / download: Direct client-to-node, FPCC verified on both ends. - avid-disperse / avid-retrieve: Full echo/ready consensus among servers.

5. Design Decisions

No external libraries. The erasure coding, finite field math, fingerprinting, hashing, and networking are all hand-written. We used `java.security.MessageDigest` for SHA-256, `java.net.ServerSocket` for TCP, and `CompletableFuture` for parallelism — all part of the standard library.

Plain TCP sockets instead of gRPC or HTTP. Simpler to debug, easier to simulate failures (just kill the process), and no dependency on protobuf or a web framework.

Systematic code. The first 4 fragments are the raw data chunks. If all nodes are up, you can skip decoding entirely.

Verification at both ends. The storage node checks FPCC on upload (catches a buggy client). The client checks FPCC on download (catches a corrupted node). Defense in depth.

Failures are uniform. A crashed node and a corrupted fragment both result in the same thing: `present[i] = false`. The decoder handles both the same way.

The FPCC seed comes from the hashes. We derive the fingerprint key as SHA-256 of all the fragment hashes. This binds the fingerprint to the specific fragments — you can't swap in a different fragment that happens to match under a different key.

6. Implementation Summary

The project is split into three Maven modules:

Module	What it does	Files	Lines
common	Wire protocol, GF(2 ⁸) arithmetic, SHA-256 utilities, fingerprinting	4	383
client	Encoding, decoding, FPCC, manifest, CLI, upload/download engine	7	1,167
storage-node	TCP server, fragment storage, AVID-FP state machine	1	397
Total		12	1,947

Key files and what they do:

- **GaloisField256.java** — All GF(2⁸) operations: add, multiply, inverse, power, matrix inversion.
- **LinearErasureCodec.java** — Builds the Vandermonde coding matrix, encodes files into fragments, decodes from any k=4 survivors.
- **HomomorphicFingerprint.java** — Polynomial evaluation fingerprinting (Horner's rule at random GF(2⁸) points derived from the seed).
- **FingerprintedCrossChecksum.java** — Creates and verifies FPCCs. The `verifyFragment()` method runs both the hash check and the fingerprint check.
- **Protocol.java** — Wire format constants and serialization helpers for all message types (STORE, RETRIEVE, DISPERSE, ECHO, READY).
- **StorageNode.java** — Multi-threaded TCP server. Handles basic STORE/RETRIEVE plus the full AVID-FP consensus state machine (echo sets, ready sets, delivery futures).
- **ErasureClient.java** — Orchestrates parallel upload (with per-fragment FPCC data) and parallel download (with FPCC verification).
- **AvidFpClient.java** — Client-side AVID-FP: disperse with quorum wait, retrieve with FPCC agreement check.
- **ShardManifest.java** — Persists the manifest (file ID, node addresses, coding params, full FPCC) as a simple key-value text file.
- **NodeConnection.java** — Thin TCP socket wrapper for talking to a single storage node.
- **Client.java** — CLI entry point with upload, download, demo, avid-disperse, avid-retrieve sub-commands.
- **HashUtil.java** — SHA-256 wrapper around `java.security.MessageDigest`.

Build system: Maven multi-module POM. Each deployable module uses `maven-assembly-plugin` to produce a standalone fat JAR. Build with `mvn clean package -q`.

7. Testing and Results

We have an automated demo script (`scripts/demo.sh`) that builds the project, starts a 6-node cluster, and runs 4 end-to-end tests:

Test 1 — Happy path. Upload a file, download it back, compare bytes. Validates that encoding, FPCC, distribution, retrieval, verification, and decoding all work correctly end to end.

Test 2 — Node failure. Kill 2 of the 6 nodes, then download. The system detects the 2 unreachable nodes, reconstructs from the remaining 4 fragments, and produces the original file.

Test 3 — Corruption detection. Upload a file, then overwrite one fragment on disk with garbage. On download, the FPCC catches the corrupted fragment (SHA-256 mismatch), marks it as an erasure, and reconstructs from the 5 healthy ones.

Test 4 — AVID-FP consensus. Restart the cluster in AVID mode with peer addresses. Disperse a file using the full echo/ready protocol, then retrieve it. Validates the entire server-to-server consensus flow.

All 4 tests pass.

Output: Upload and Download

The screenshot below shows the build, cluster startup, and a successful upload. The file (176 bytes) is encoded into 6 fragments of 44 bytes each, the FPCC is computed, and all fragments are uploaded in parallel to the 6 nodes.

```

$ bash scripts/demo.sh 2>&1

=====
Building the project
=====

Build successful.
Created test file (176 bytes).

=====
TEST 1: Upload and Download (happy path)
=====

[cluster] Starting 6 nodes (AVID=false)...
Node 0: port=7100
Node 1: port=7101
Node 2: port=7102
Node 3: port=7103
Node 4: port=7104
Node 5: port=7105

[cluster] Waiting for nodes...
OK localhost:7100
OK localhost:7101
OK localhost:7102
OK localhost:7103
OK localhost:7104
OK localhost:7105

Cluster is up!
Stop with: kill $(cat /tmp/objectstore-pids.txt)
Nodes: localhost:7100 localhost:7101 localhost:7102 localhost:7103 localhost:7104 localhost:7105

Uploading...

=====
Uploading with FPCC Integrity Verification
=====

File   : /Users/sanjithjayasankar/Documents/gitrepo/Self-Healing-Distributed-Object-Store/testfile.txt
FileId : myfile
Codec  : (4 data + 2 parity = 6 total)

[1/3] Read 176 bytes.
[1/3] Encoded → 6 fragments of 44 bytes each.
[2/3] FPCC computed.
[3/3] Uploading 6 fragments in parallel...
✓ fragment 3 → localhost:7103
✓ fragment 5 → localhost:7105
✓ fragment 0 → localhost:7100
✓ fragment 2 → localhost:7102
✓ fragment 4 → localhost:7104
✓ fragment 1 → localhost:7101

✓ Upload complete.
=====

```

Output: Node Failure Recovery

After killing nodes on ports 7104 and 7105, the download detects both as unreachable (Connection refused), retrieves the remaining 4 fragments, verifies them against the FPCC, and reconstructs the original 176-byte file.

```

=====
TEST 2: Node failure recovery (kill 2 of 6 nodes)
=====

Killing nodes on ports 7104 and 7105...
Downloading with 2 nodes down...
Loading manifest from /Users/sanjithjayasankar/Documents/gitrepo/Self-Healing-Distributed-Object-Store/myfile.manifest

=====
Downloading with FPCC Integrity Verification
=====
FileId : myfile
Codec  : (need 4 of 6 fragments)

⚠ fragment 4 - UNREACHABLE (Connection refused)
⚠ fragment 5 - UNREACHABLE (Connection refused)
✅ fragment 1 - OK (44 bytes)
✅ fragment 0 - OK (44 bytes)
✅ fragment 3 - OK (44 bytes)
✅ fragment 2 - OK (44 bytes)

Retrieved 4 / 6 fragments.
Reconstructing from healthy fragments...

✅ Wrote 176 bytes to 'recovered.txt'
=====

```

8. Project Structure

```

distributed-object-store/
├── pom.xml                # Root Maven POM
├── README.md
├── REPORT.md
├── common/
│   ├── pom.xml
│   └── src/main/java/com/objectstore/common/
│       ├── GaloisField256.java    # GF(2^8) arithmetic
│       ├── HashUtil.java          # SHA-256 utilities
│       ├── HomomorphicFingerprint.java # Polynomial evaluation fingerprinting
│       └── Protocol.java          # Wire protocol
├── client/
│   ├── pom.xml
│   └── src/main/java/com/objectstore/client/
│       ├── Client.java            # CLI entry point
│       ├── ErasureClient.java     # Upload/download engine
│       ├── AvidFpClient.java      # AVID-FP client
│       ├── LinearErasureCodec.java # (6,4) Vandermonde erasure codec
│       ├── FingerprintedCrossChecksum.java # FPCC
│       ├── ShardManifest.java     # Manifest persistence
│       └── NodeConnection.java    # TCP socket wrapper
├── storage-node/
│   ├── pom.xml
│   └── src/main/java/com/objectstore/node/
│       └── StorageNode.java       # TCP server + AVID-FP
├── scripts/
│   ├── demo.sh                 # Automated 4-test demo
│   └── start-cluster.sh        # Launches 6-node cluster

```